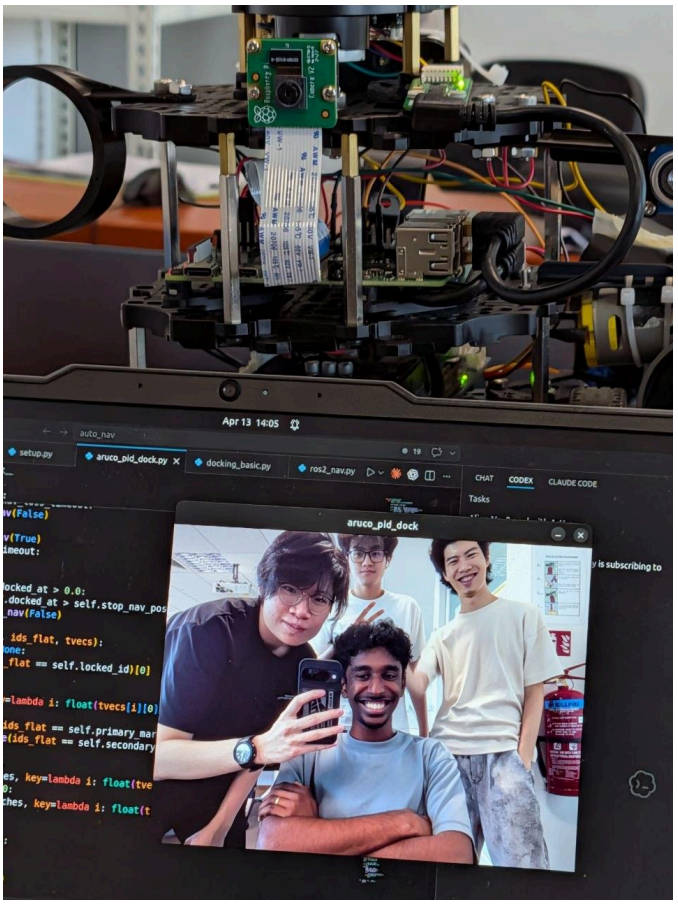
CDE2310 FINAL REPORT G2

AY2025/26 Sem2 Group 8

Team member	Student ID
HSU YUCHEN	A0312064U
Jonathan Sugiharto Santosa	A0303274M
Lim Kai Ler, Ethan	A0307499M
Ramanathan Alagu Subramaniam	A0334850E



Content List

1. Requirement Specifications.... 1
    1.1 Background.... 1
    1.2 Problem Statement.... 1
    1.3 Project Objectives.... 1
    1.4 Functional Requirements.... 1
    1.5 Non-Functional Requirements.... 2
    1.6 Design Constraints.... 2
2. Concept of Operations (Con-Ops).... 2
    2.1 Operational Overview.... 2
    2.2 Operational Sequence.... 2
    2.3 Key Actors.... 3
    2.4 Operational Assumptions.... 3
3. High-Level Design.... 4
    3.1 System Architecture Overview.... 4
    3.2 Functional Block Description.... 5
    3.3 Design Rationale.... 6
4. Subsystem Design.... 7
    4.1 Navigation and Mapping Subsystem.... 7
    4.2 Perception Subsystem.... 9
    4.3 Payload Storage and Feeding Subsystem.... 10
    4.4 Launcher Subsystem.... 13
    4.5 Electrical and Control Subsystem.... 15
5. Interface Control Documents (ICD).... 16
    5.1 Hardware Interfaces.... 16
        5.2 Software Interfaces.... 17
    5.3 Electrical Compatibility Notes.... 18
6. Software / Firmware Development Documentation.... 18
    6.1 Software Architecture.... 18
    6.2 Main Control Logic.... 19
    6.3 Navigation Software.... 20
    6.4 Payload Control Software.... 20
    6.5 Firmware / Hardware Integration.... 20
    6.6 Error Handling.... 20
7. Real World Testing Documentation.... 21
    7.1 Testing Objectives.... 21
    7.2 Subsystem Tests.... 21
        Auto Mapping of Maze:.... 21
        ArUco Docking:.... 21
        Payload Delivery:.... 22
        Integrated Mission Test:.... 22
    7.3 Performance Metrics.... 22
    7.4 Common Failure Modes to Monitor.... 23
    7.5 Final Run Lessons Learnt.... 23
        Mechanical redesign:.... 23
        Electrical System Improvement:.... 24
        Software Drawbacks:.... 24
8. Appendix.... 26
    Appendix A: Mechanical Drawing of Design.... 26
    Appendix B: Electrical Schematic Drawing of Design.... 28

Github: https://github.com/Kailer811/r2auto_nav_CDE2310

Final run Video : https://youtu.be/lXcatRy6V4A

1. Requirement Specifications

1.1 Background

Autonomous Mobile Robots (AMRs) play an increasingly important role in modern warehouse intralogistics. By automating material transport and improving the flow of information between workstations, AMRs improve operational efficiency, reduce human workload, and enhance delivery consistency. In warehouse-like environments, AMRs are expected to navigate dynamically, respond to changing obstacles, and perform precise delivery actions while maintaining reliable localisation and mapping capabilities.

1.2 Problem Statement

To function effectively in a warehouse environment, an AMR must be capable of three core tasks: autonomous navigation in an unknown environment, dynamic path planning with obstacle avoidance, and accurate payload delivery at designated stations. In this project, these real-world challenges are simulated using a TurtleBot3 platform operating in an unknown maze that represents a warehouse. The robot must enter the environment, generate its own map, localise within the map, identify delivery stations, transport payloads, and complete unloading tasks without human intervention after mission start.

1.3 Project Objectives

The main objective of this project is to design and implement an integrated AMR system that is able to:

- autonomously explore and map an unknown maze using SLAM,
- localise itself robustly during operation,
- identify and approach delivery stations,
- plan efficient paths while avoiding obstacles,
- deliver payloads accurately using a compact launcher subsystem,
- coordinate sensing, computation, and actuation through an embedded control architecture.

1.4 Functional Requirements

The robot shall:

1. enter an unknown maze environment and begin autonomous exploration;
2. build a usable map of the environment using onboard sensing;
3. localise itself continuously while navigating;
4. detect and identify target delivery stations;
5. plan collision-free paths to target locations;
6. avoid static and dynamic obstacles during navigation;

7. dock or align sufficiently with the target station before unloading;
8. Release or launch payloads reliably into the designated target zone;
9. Repeat the above process until all assigned objectives are completed.

1.5 Non-Functional Requirements

The system should also satisfy the following performance requirements:

- Reliability: mapping, navigation, and launching should remain repeatable across multiple trials;
- Compactness: the payload mechanism must fit within the TurtleBot3 dimensional constraints;
- Stability: the payload storage and launching subsystem must resist disturbances caused by robot motion;
- Responsiveness: sensing and control loops should operate with sufficiently low latency for practical decision-making;
- Safety: the robot should avoid collisions with walls, obstacles, and mission structures;
- Maintainability: the system architecture should be modular enough for subsystem-level debugging and replacement.

1.6 Design Constraints

The design is constrained by:

- the limited payload and mounting space available on TurtleBot3,
- restricted electrical power and voltage compatibility across components,
- sensor limitations such as camera lighting sensitivity and ultrasonic measurement noise,
- the lightweight and bounce-prone nature of ping pong balls,
- The requirement to operate in an unknown maze without preloaded environment maps.

2. Concept of Operations (Con-Ops)

2.1 Operational Overview

The AMR begins operation from a designated start point and enters an unknown warehouse-like maze. Its onboard LiDAR and camera systems are activated immediately to support environment sensing and target detection. The robot first performs autonomous exploration and map generation using SLAM. As the environment becomes progressively mapped, the robot evaluates reachable regions, detects potential objectives, and prioritises task completion based on current mission status.

Once a target is identified, the robot generates a path to the station while avoiding obstacles. Upon approaching the target it executes a calculated maneuver to ensure precise docking. When the robot has docked, the payload subsystem is triggered to deliver the ping pong ball payload to the target location. This process repeats until all mission objectives are completed.

2.2 Operational Sequence

The mission sequence can be summarised as follows:

1. System initialization:
   The Raspberry Pi, OpenCR, LiDAR, camera, ultrasonic sensor, motors, and launcher subsystem are powered on and checked.
2. Exploration and mapping:
   The robot uses a Frontier algorithm to determine the best unexplored area to path towards. Nav2 then plots the shortest path while Cartographer generates the map of the unknown environment.
3. Objective detection:
   Nearby objectives are identified using a Raspberry Pi camera by detecting their corresponding ArUco markers. Their positions are then localized on the Cartographer map, allowing Nav2 to generate a path toward them.
4. Final alignment:
   When the robot is 20cm away from the ArUco and has a visual of it. It switches to precise docking. The robot detects the ArUco marker, turns to move parallel to the wall it is mounted on, and continues until the marker is directly beside it. It then turns to face the marker and moves straight toward it.
5. Payload delivery:
   The servo-controlled feed mechanism meters one ball into the launcher with timing, and the flywheel subsystem shoots the payload to the station.
6. Payload delivery modified
   We use an additional ultrasonic sensor to sense the target being in frame. It scans and ensures that the target is in line with the payload delivery before firing
7. Mission update
   The control system marks the objective as complete and selects the next target or unexplored region.
8. Mission completion
   The robot stops only after all objectives are completed, or no feasible objective remains, and the map has been fully mapped.

2.3 Key Actors

- Robot platform: TurtleBot3 chassis with onboard controllers and sensors
- Control unit: Raspberry Pi and OpenCR
- Sensors: LiDAR, camera, ultrasonic sensor
- Actuators: Drive motors, flywheel motor, servo motor
- Environment: Maze with walls, stations, and obstacles
- Operator: 1 member of the team initialises the mission and supervises testing only

2.4 Operational Assumptions

The concept of operations assumes that:

- The robot starts from a known initial pose,
- The environment is unknown but traversable,
- The target station geometry is detectable using onboard sensors,
- All electrical and mechanical components are functioning normally before deployment,
- The robot receives no manual assistance after mission start.

3. High-Level Design

3.1 System Architecture Overview

The Autonomous Mobile Robot (AMR) is designed as an integrated cyber-physical system that combines sensing, computation, decision-making, and mechanical actuation into a single coordinated platform. At the highest level, the robot architecture can be divided into three major functional layers: the perception layer, the decision-making layer, and the execution layer. These layers work together continuously during operation so that the robot can sense its surroundings, interpret mission conditions, and carry out the required actions in real time.

The perception layer is responsible for collecting information about both the environment and the delivery objectives. This layer consists primarily of the LiDAR, camera, and ultrasonic sensor. The LiDAR provides environmental geometry that allows the robot to detect walls, free space, and obstacles, making it essential for map generation and localisation. The camera is used for visual identification of targets, markers, or delivery stations, allowing the robot to distinguish mission objectives from the surrounding environment. The ultrasonic sensor provides short-range distance measurements and timing-related data, which are especially useful for tasks involving moving targets or close-range triggering. Together, these sensors provide complementary information so that the robot is not dependent on only one sensing modality.

The decision-making layer forms the computational core of the system. Its role is to process incoming sensor data and convert that information into meaningful mission actions. This layer performs the key software functions required for autonomous operation, including simultaneous localisation and mapping (SLAM), exploration of unknown space, target detection, path planning, navigation control, and payload firing logic. By continuously updating the robot's understanding of the environment and its mission status, this layer determines what the robot should do next at each stage of operation. For example, it decides whether the robot should continue exploring, move toward a target, perform final alignment, or trigger the payload launcher.

The execution layer is responsible for physically carrying out the decisions made by the software. This includes robot movement through the TurtleBot drive motors as well as payload handling through the launcher motor and servo-actuated feeding mechanism. Once commands are generated by the control logic, this layer converts those commands into real mechanical actions, such as moving forward, turning, stopping, aligning with a station, or launching a ping pong ball.

At the centre of this architecture is the Raspberry Pi, which acts as the main computational and coordination unit of the AMR. It receives data from the sensors, processes mission logic, runs the navigation and launching algorithms, and sends the

corresponding commands to the relevant hardware components. In this sense, the Raspberry Pi serves as the brain of the robot, coordinating the interaction between perception, planning, and actuation. Supporting this is the OpenCR board, which provides the motor-level interfacing required for the TurtleBot3 mobility platform. While the Raspberry Pi handles high-level computation and decision-making, the OpenCR enables reliable low-level communication and execution for robot motion control. This division of responsibility improves system organisation and allows the robot to maintain both computational flexibility and stable mobility control.

Overall, the architecture is designed to support autonomous operation in an unknown maze-like warehouse environment, where the robot must repeatedly sense, decide, and act in a closed feedback loop until all payload delivery objectives are completed.

3.2 Functional Block Description

At a high level, the operation of the AMR can be represented by the following functional flow:

Sensors → Raspberry Pi → Motion / Launcher Commands → Environment Interaction

This flow summarises how information moves through the system and how that information leads to physical action. The robot begins by collecting data through its onboard sensors. These raw sensor inputs are then transmitted to the Raspberry Pi, where they are interpreted and processed according to the mission logic. Based on the resulting decisions, the Raspberry Pi sends appropriate commands to the movement and payload subsystems. These actions then change the robot's interaction with the environment, generating new sensor data and continuing the feedback cycle.

The LiDAR is responsible for providing geometric information about the surroundings. It scans the environment and supplies range data that can be used for obstacle detection, map generation, and localisation. This data is critical for SLAM and for constructing the occupancy grid that the robot uses to distinguish explored free space, obstacles, and unknown regions.

The camera provides visual information for recognising targets, stations, or fiducial markers. While LiDAR is useful for understanding spatial layout, it does not identify the meaning of an object. The camera complements this by helping the robot determine whether a detected feature corresponds to a valid mission objective. This is especially important during the final approach and delivery stages, where visual confirmation improves targeting confidence.

The ultrasonic sensor provides short-range proximity information. Its primary role is to measure distances in close-range scenarios or to support timing-based decisions when interacting with moving targets. In these situations, the robot can use distance-over-time measurements from the ultrasonic sensor to estimate motion and determine when a launch command should be executed.

Once sensor information is collected, the Raspberry Pi processes this data as the main control centre. It fuses the incoming information with the current map, localisation estimate, and mission state. Based on this combined understanding, it decides whether the robot should continue exploration, replan its route, approach a detected target, stop for alignment, or trigger the launcher. This makes the Raspberry Pi the key link between sensing and action.

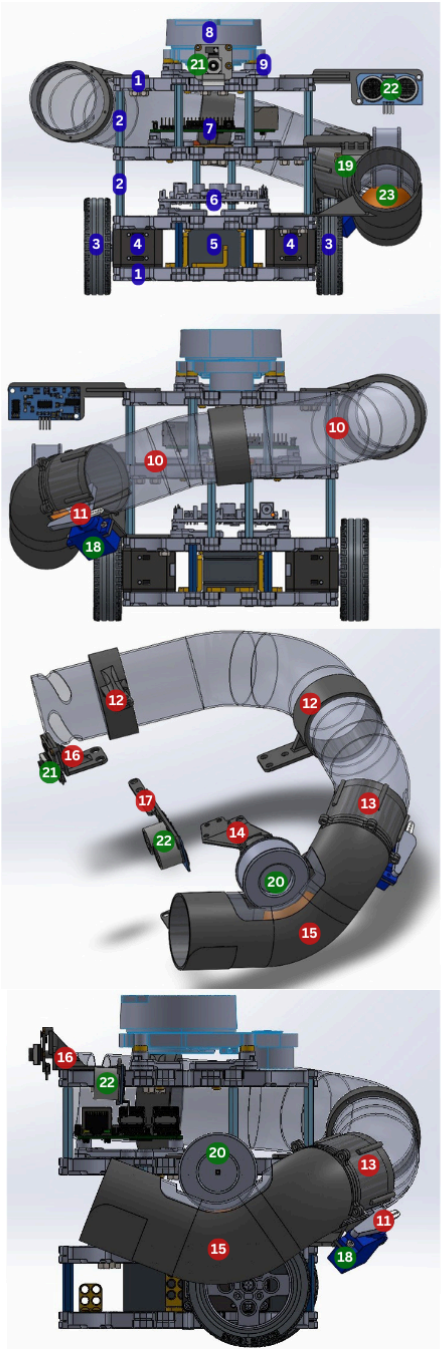
After the decision has been made, commands are issued to the relevant hardware. Navigation-related commands are sent to the TurtleBot drive motors, enabling the robot to move, turn, slow down, or stop as required. Payload-related commands are sent

to the launcher subsystem, which includes the flywheel motor and servo latch mechanism. These commands ensure that the ball is first metered into position and then launched only when the robot has reached the correct firing condition.

The final stage of the block flow is environment interaction. In response to the issued commands, the robot moves through the maze, avoids obstacles, approaches delivery stations, and performs payload delivery. These actions in turn alter the robot's surroundings and internal state, producing updated sensor readings that feed back into the system. As a result, the robot operates in a continuous closed-loop cycle rather than as a fixed one-time sequence.

This functional block structure is valuable at the high-level design stage because it clearly shows the relationship between sensing, computation, and actuation without yet going into subsystem-specific implementation detail. It also highlights that successful mission completion depends on the coordination of all blocks rather than on any one part alone.

3.3 Design Rationale



1. TurtleBot chassis frame (stacked plates)
2. Plate support pillars
3. Wheel and Tire
4. 2x Dynamixel DC gear motors
5. Lipo Battery
6. Opencr1.0
7. Raspberry Pi
8. LDS
9. USB2LDS
10. Aluminium wire framework
11. Elbow bend Gate for Servo extention
12. Tube Mount clamp middle and end
13. Servo and shooting pipe Mounting bracket
14. DC motor Mounting bracket
15. Front shooting pipe
16. Rpi Camera Bracket
17. Ultrasonic sensor bracket
18. Servo motor
19. DC motor
20. Flywheel wheel
21. Rpi Camera
22. Ultrasonic sensors
23. Mission-critical payload module (PingPong)
24. M4 Hex screw and M4 Nut + M2 cross screw and M2 nut

The high-level design of the AMR was developed with three main priorities in mind: compactness, adaptivity, and reliability. These priorities arise from the prototype style of the project as well as the many adjustments needed to integrate perfectly with the software to execute the mission.

The first priority is compactness. This is important as the robot has space limitations from the height and strength of the motor; we want to keep the robot's COG low to keep its movement stable. This height barrier forces me to think of a design that is compact and modular. The final design is modular as it has fixed brackets that can move vertically depending on the height of the pillar supports, which in the end we increased by 30mm to increase storage and clearance for the electronics. The system still works because the storage system uses Aluminum wire to guide the balls down. We used a slightly stronger motor (R140

DC motor), which has a larger range for docking. This is to account for the random error of the distance due to the docking system. The system is able to dock with a range from 2cm to 20cm. The robot operates on a limited chassis with restricted space for mounting bulky payload mechanisms. Instead, each component must be integrated efficiently into the overall structure. This consideration strongly influenced choices such as not using a vertical loading launcher.

The second priority, adaptivity, supports both development and maintenance. By structuring the robot into distinct parts that can be connected together with a malleable structure, such as the aluminium wire. The design makes it easier to test, debug, and improve one part of the system without redesigning the entire robot. Another example of the is the mount for the DC motor, i added slots for zipties as well as room for foam tape to give tension and increase friction between the motor and the bracket, the tension from the thick tape presses the motor down so that the distance between the flywheel and the chamber is too small for a pingpong ball, hence when a pingpong ball is pulled inside it will force itself and push the motor slightly upwards creating more pressure for the flywheel to give power to the launch. This modular arrangement is particularly valuable during prototyping, when we realise the software perception and motors are flipped and fixing it in code would be very troublesome, hence I reassembled the robot mechanically and moved around the motors to match the software's perceived locations.

The third priority, reliability, reflects the fact that the robot must operate in an unknown maze and a moving target. No single subsystem can guarantee mission success on its own. Strong navigation without reliable feeding still leads to failure. Likewise, a good launcher is not useful if the robot cannot localise or align properly. For this reason, most of the design revolves around variable but lockable adjustments. As for the mechanical parts, I used a tight screw hole to minimise the movement and reduce the instability, and used zip ties to lock the position of critical parts such as the launching chamber and the DC motor holder. The system improves delivery reliability through a combination of an adjustable and reliable mechanical system.

Overall, the chosen high-level design provides a practical balance between system capability and implementation feasibility. It allows the AMR to operate autonomously in an unknown maze, respond to sensor feedback in real time, and coordinate movement with payload delivery in a controlled and repeatable way. This makes it well-suited to the objectives of the project while remaining flexible enough for continued refinement through testing and iteration.

4. Subsystem Design

4.1 Navigation and Mapping Subsystem

Functions

The main functions of this subsystem are to:

- perform SLAM in an unknown environment,
- generate and update an occupancy grid map,
- estimate the robot's pose in real time,
- identify unexplored frontiers for autonomous exploration,
- compute feasible routes to selected frontiers using the nav2 stack,
- avoid collisions while moving through the maze,

- support the final approach and docking behaviour near the target station.

Design Approach

The subsystem is built around a LiDAR-based SLAM framework, which allows the robot to simultaneously construct a map of the environment and estimate its own position within that map. As the robot moves through the maze, LiDAR scan data is continuously collected and processed to detect surrounding walls, obstacles, and free space. This information is then used to update an occupancy grid map, where each region of space is represented as free, occupied, or unknown. The occupancy grid serves as the primary internal map representation for navigation and mission planning.

To ensure efficient exploration of the unknown maze, the robot uses a frontier-based exploration strategy. In this method, the robot identifies frontier regions, which are the boundaries between explored and unexplored space, and selects them as candidate destinations. This allows the robot to progressively expand its map coverage in a structured way instead of wandering randomly through the environment. Frontier-based exploration is particularly suitable for this project because it provides a simple and effective way to achieve broad map coverage without requiring prior knowledge of the maze layout.

Once a target or objective location has been identified, the subsystem transitions from general exploration to directed navigation. At this stage, A* path planning is performed on the occupancy grid to compute a feasible route from the current robot position to the desired goal. A* was chosen because it is computationally manageable, works well with grid-based maps, and can produce efficient collision-free paths in structured indoor environments. The path planner uses the existing map to avoid known obstacles while guiding the robot toward the target location.

During path execution, local obstacle avoidance is applied to handle unexpected obstructions or deviations that are not fully captured in the global path plan. This is important because even with a valid global route, the robot may still encounter local disturbances, uncertainty in pose estimation, or temporary obstacles in its path. By combining global planning with local correction, the subsystem improves both navigation safety and mission robustness.

The navigation and mapping subsystem is closely linked to the perception and payload subsystems. The perception subsystem supplies the environmental data required to build the map and detect targets, while the navigation subsystem places the robot in the correct region and orientation for the payload system to complete the delivery. As such, this subsystem not only supports mobility but also directly affects the overall success of the mission.

Overall, the navigation and mapping subsystem is designed to provide reliable autonomous movement, structured exploration, and goal-directed routing in an unknown maze. Its role is central to mission completion because accurate payload delivery can only occur if the robot is first able to navigate to the appropriate target position in a stable and repeatable way.

4.2 Perception Subsystem

The perception subsystem is responsible for sensing the robot’s surroundings and providing the information required for mapping, localisation, target recognition, and launch timing. In the overall AMR system, perception acts as the interface between the robot and the environment. Without reliable perception, the robot would not be able to understand where obstacles are, determine whether a target is present, or estimate when a payload should be launched. For this reason, the perception subsystem is designed as a layered sensing system that combines multiple sensing modalities rather than relying on a single source of information.

Functions

The main functions of this subsystem are to:

- detect walls, obstacles, and free space,
- provide environmental data for SLAM and localisation,
- identify targets, markers, or delivery stations,
- support close-range alignment and docking,
- estimate short-range target distance,
- provide timing information for moving-target launch tasks,
- improve mission robustness through complementary sensing.

Design Approach

The perception subsystem consists primarily of the LiDAR, camera, and ultrasonic sensor, each of which serves a different but complementary role. Together, these components provide the robot with both geometric and task-specific information.

The LiDAR serves as the primary sensing component for map building and localisation. It provides range measurements of the environment, allowing the robot to detect walls, corners, openings, and obstacles. These measurements are essential for SLAM, as they allow the robot to construct its occupancy grid map and continuously estimate its position relative to the environment. Because the navigation subsystem depends heavily on this information, the LiDAR forms the backbone of the robot's environmental perception.

The camera provides visual information that cannot be directly inferred from LiDAR data alone. While LiDAR is excellent for measuring geometry, it cannot determine whether a particular object corresponds to a valid target or marker. The camera, therefore, plays a key role in recognising objectives such as fiducial markers or delivery stations. It also supports final-stage alignment when the robot needs more task-specific information than simple geometric ranging. This is particularly useful during docking or payload delivery, where target identity and relative orientation become more important than broad environmental mapping.

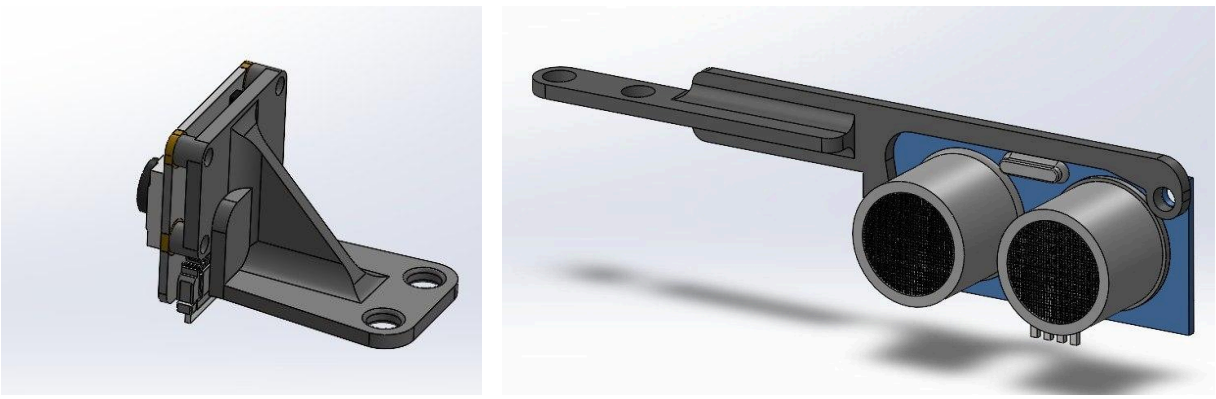


Figure 1: Rpi camera for ArUco detection (right) and ultrasonic sensor for moving target detection (left)

The ultrasonic sensor is used primarily for short-range distance measurement and timing-related tasks. Its role is especially relevant in scenarios involving moving targets, where the robot must determine when the target will be in the correct position for launch. By collecting distance-over-time information, the ultrasonic sensor allows the system to estimate target motion and trigger the launcher at an appropriate moment. Even though it is a simpler sensing device compared with LiDAR or a camera, it provides a valuable source of direct short-range information that is well-suited to timing-based decisions.

A key feature of the design is that these sensors are used in combination rather than isolation. LiDAR supports map construction and navigation, the camera supports recognition and alignment, and the ultrasonic sensor supports close-range timing and motion estimation. This layered approach improves system robustness by reducing overdependence on any one sensor. For example, if camera performance is affected by lighting conditions, the robot can still rely on LiDAR for navigation. Likewise, if LiDAR is not sufficient for a moving-target timing task, the ultrasonic sensor provides a more direct measurement at close range.

The perception subsystem is tightly coupled with both the navigation subsystem and the payload delivery subsystem. Navigation depends on perception for environmental awareness, while payload delivery depends on perception for target recognition, alignment, and launch timing. As a result, the performance of this subsystem has a direct influence on the overall accuracy and reliability of the AMR.

The figure above presents the final bracket design for mounting the RPi Camera and ultrasonic sensor. Relative to the prototype, the design was improved by updating the hole tolerances, increasing the thickness of the printed parts, and adding support ribs to enhance structural rigidity. In addition, the mounting position and sensing angle of both the ultrasonic sensor and Raspberry Pi Camera were adjusted based on prototype testing to improve alignment and stability.

Overall, the perception subsystem is designed to provide the robot with the situational awareness needed for autonomous operation. By combining sensing methods with different strengths, it supports reliable mapping, target detection, docking, and launch triggering throughout the mission.

4.3 Payload Storage and Feeding Subsystem

The payload storage and feeding subsystem is responsible for holding the ping pong balls securely during robot operation and delivering them one at a time into the launcher in a controlled and repeatable manner. Since the robot must navigate an unknown maze while accelerating, decelerating, and turning, the payload cannot simply be placed in an open container. Ping pong balls are lightweight, easily displaced by motion, and highly prone to bouncing or shifting position. There is also the problem of double-feeding and balls getting stuck. As a result, the storage and feeding mechanism must not only carry the payload, but also maintain order; hence, we chose a tube as a reliable storage. We used an L gate on a servo motor that rotates 90 degrees, as we need a reliable single-shot mechanism to fulfil the timing requirements of the mission.

Functions

The main functions of this subsystem are to:

- hold all payload balls securely during robot motion,
- prevent free rolling, bouncing, and accidental discharge,
- guide the balls along a controlled path toward the launch region,

- feed exactly one ball into the launching position at a time,
- minimise jamming, misalignment, and double-feeding,
- maintain reliable operation despite the vibration and movement of the robot.

Design Approach

To satisfy these requirements, the design adopts a closed-loop tube reservoir that stores all ping pong balls within a continuous guided path. Unlike an open tray or box, the closed-loop geometry physically constrains the balls and prevents them from scattering when the robot changes speed or direction. This is especially important because even small disturbances can cause lightweight balls to shift unpredictably, which would reduce feeding consistency and compromise the launch sequence.

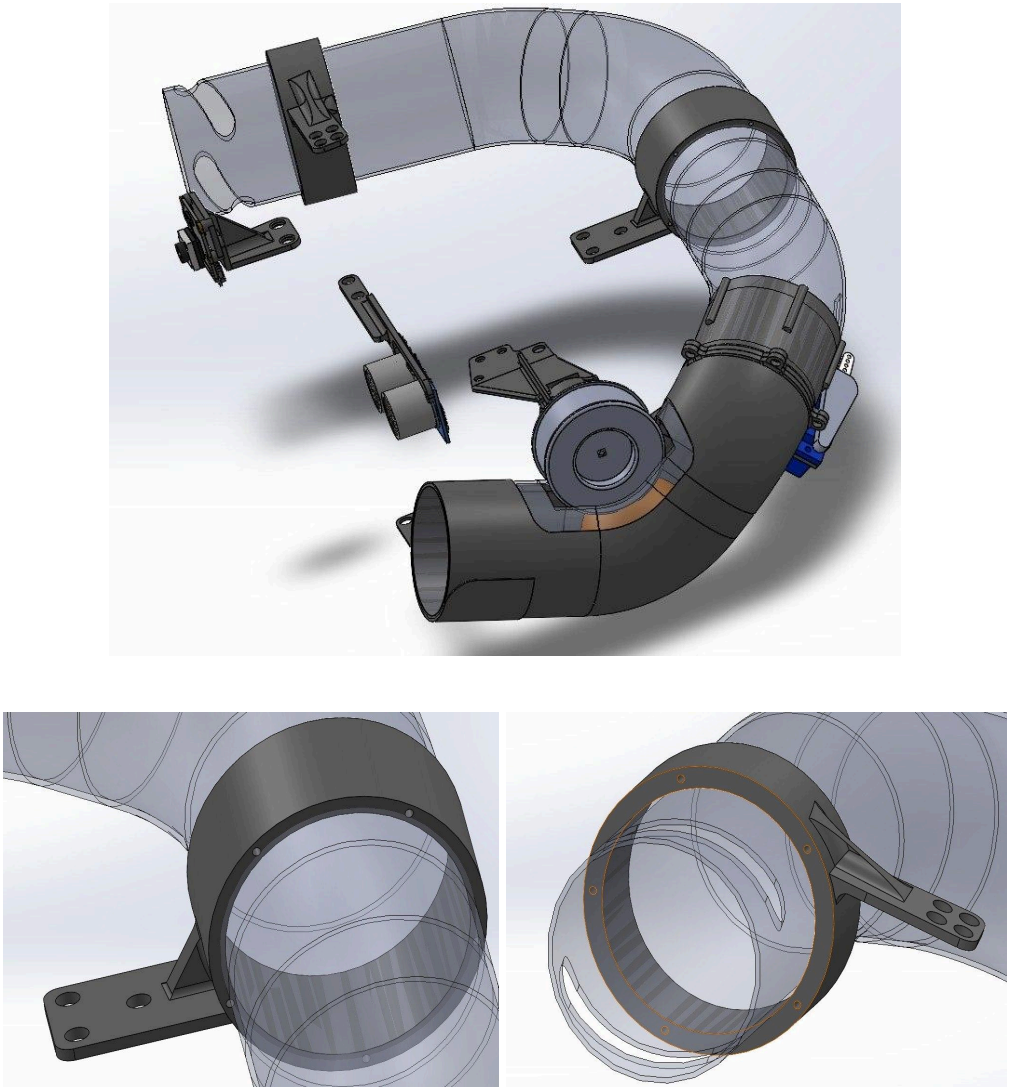


Figure 2: payload and storage subsystem overview

The tube reservoir also serves as both a storage space and a feed path. Instead of requiring a separate hopper and transfer stage, the balls remain within the same guided channel from storage to firing. This reduces the number of transitions between mechanical sections, which in turn helps lower the risk of misalignment and jams. The looped geometry is also smooth, compact and space-efficient, making it more suitable for integration onto the TurtleBot3 platform, where mounting volume is limited. The transparent tube in the image above is a substitute for the 5-wire aluminium wire system we used in our final

design. We used aluminium wires as our final tube instead of buying tubes online or using a printed tube because aluminium wires are significantly lighter and it is malleable while still sturdy enough to direct the ping pong balls and will keep its shape if accidentally bumped or hit a wall.

I designed the brackets with ribs and increased the thickness of the walls from the prototype version to create a more sturdy system after I found out that 1mm wall thickness feels like a potato chip. I also added bigger holes for the 1mm aluminium wire to pass through; originally, my prototype had it at 1.2mm, which would be inconsistent; some of the holes are tight, while some are nonexistent. I increased the hole diameter to 1.8mm in my newest design, and the fit is now perfect; some holes have slight resistance, some are smooth. I also fixed the Hole tolerance for the screw, at first i didnt put extra tolerance (40mm diameter) to all of the holes for all of the brackets, all of them were too small for a 40mm screw, i had to use 30mm screw on my first prototype, this cause all of the brackets to be a bit shaky and can move around, i ziptied them to lock the orientation but it wasn't sustainable, hence in my new iteration i made sure to include an extra 10% tolerance for the hole diameter. This solves the problem, and we no longer use zip ties to mount the brackets to the plates.

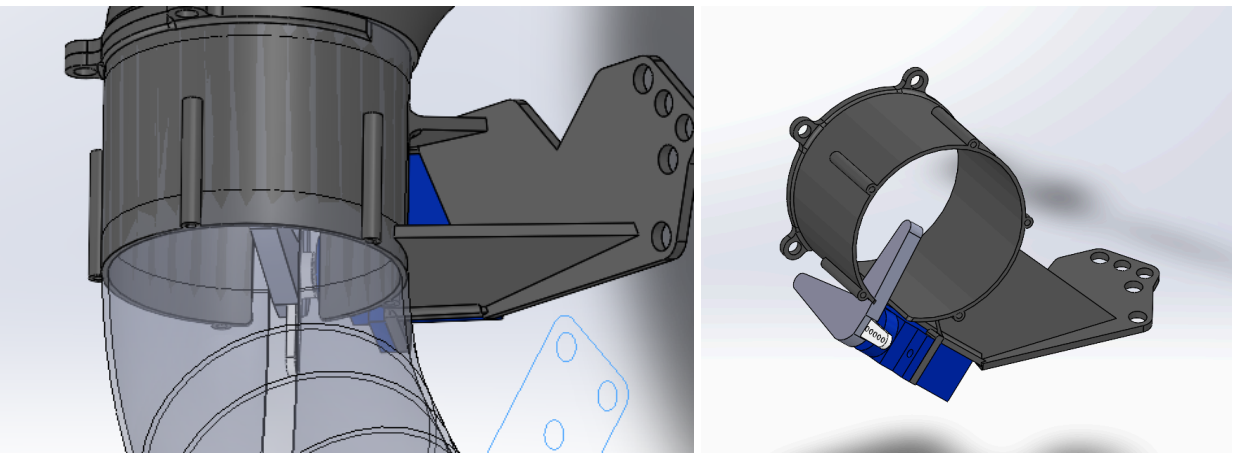


Figure 3: Feeding layout overview

At the end of the feed path, a servo-driven L-shaped latch is used as the metering mechanism. The latch rotates through a slit in the pipe and selectively controls the movement of the balls. During operation, it temporarily holds back the remaining stack while allowing only one ping pong ball to move into the firing position. Once that ball is chambered, the latch returns to its blocking position to stop the next ball from entering prematurely. In this way, the latch acts as a simple but effective gate for single-ball feeding. I CAD the L latch to perfectly press fit to the factory-made servo horn, as the 3d print can't accurately mimic the small output gear of the servo.

This approach was selected over earlier push-based concepts that depended on larger vertical drop distances or more complicated transfer arrangements. Those earlier concepts were less suitable because the available space on the TurtleBot3 is limited, particularly in the vertical direction. By using a rotating latch integrated directly into the feed path, the current design reduces spatial requirements while still maintaining good control over ball sequencing.

Another important aspect of this subsystem is its direct interaction with robot motion. Because the robot moves dynamically while carrying the payload, the storage mechanism must perform reliably even under vibration, turning, and changing speed. The guided nature of the reservoir helps stabilise the payload during these conditions, ensuring that the ball order remains controlled and that the feed position is predictable when a launch command is issued.

The payload storage and feeding subsystem also acts as the link between transport and launch. It ensures that the launcher does not receive balls randomly or in clusters, but only when the system is ready to fire. This controlled presentation of the payload is essential for repeatable launch performance and overall mission success.

Overall, the payload storage and feeding subsystem is designed to prioritise compactness, feed reliability, and mechanical simplicity. Its performance is critical because even if navigation and docking are successful, the mission will still fail if the robot cannot present the payload consistently to the launcher.

4.4 Launcher Subsystem

The launcher subsystem is responsible for delivering the ping pong ball payload from the robot into the target box. Its role is not simply to eject the ball, but to do so with sufficient consistency and timing that the delivery task can be completed reliably. Because the mission involves repeated launches under tight spatial and timing constraints, the launcher must provide a repeatable output while remaining compact enough to fit around the TurtleBot3 chassis.

Functions

The main functions of this subsystem are to:

- accelerate the ping pong ball to a repeatable launch speed,
- support short-range to medium-range targeted delivery into the station,
- respond quickly once the fire command is issued,
- work together with the feeding mechanism to ensure one-ball-at-a-time launching,
- maintain stable launch performance over repeated cycles.

Design Approach

The launcher uses a compact vertical flywheel mechanism designed specifically for short-range launches of approximately 20 to 40 cm. A flywheel-based design was chosen because it provides a relatively simple and lightweight way to transfer energy to the ping pong ball through frictional contact. Compared with more complex launching concepts, such as spring-loaded or multi-stage mechanisms, the flywheel launcher offers a better balance between controllability, packaging, and ease of integration with the rest of the robot.

The launcher is arranged vertically to make more efficient use of the limited chassis space. Since the TurtleBot3 has tight dimensional constraints, especially in the vertical and frontal mounting regions, the launcher geometry had to remain compact while still aligning well with the feed path and target direction. The vertical flywheel layout supports this by allowing the mechanism to be integrated closely with the circular storage loop and firing section.

Figure 4: launcher (flywheel) layout overview

The motor bracket, launcher tube, and flywheel went through several design iterations before reaching a satisfactory level of shot consistency, launch power, and launch height. In the first iteration, both the motor bracket and launcher tube were too thin, which caused significant vibration when the motor was running. This instability affected the alignment of the launcher and resulted in inconsistent shots.

To improve motor stability, the motor bracket was designed with slots for zip ties so that the clamping pressure on the motor could be adjusted and controlled. However, after the first print, it was observed that the print orientation was not ideal, and the layer lines made the bracket more prone to failure after repeated use. To address this, excess material was removed from the bracket, the part was reoriented and printed diagonally, and thicker supporting fins and a thicker base plate were added to improve strength. Similar reinforcement was also applied to the launcher tube supports by increasing the thickness of the ribs.

However, this new iteration introduced the opposite problem: the structure became too rigid compared with the earlier overly flexible design. As a result, the ball would get stuck about 10% of the time, especially when the battery voltage was lower. This happened because the motor no longer had enough torque to pull the ball through the increased contact pressure created by the bracket arrangement. The issue was eventually resolved by adding a plate to raise the motor height by approximately 1 to 2 mm, reducing the excessive compression while still maintaining sufficient contact for launch.

The selected launch range is relatively short, which means the required launch energy is low. This is advantageous because it allows the system to operate within the available motor and power constraints without requiring a large, heavy, or high-speed mechanism. Although the ball does not need to travel far, repeatability is still crucial, since slight variation in exit speed or alignment can cause the ball to miss the target or bounce out of the station.

The flywheel motor is designed to start spinning up only at 2 seconds before the moment of launch. This ensures that when the firing command is given, the launcher is already in a ready state and can respond immediately. We are want to keep the wheel at a steady RPM in order to improve consistency between launches, since each ball is launched under similar wheel-speed

conditions. This is especially useful in time-sensitive scenarios, such as firing at a moving target, where delays caused by motor acceleration would reduce accuracy.

In practice, the launcher functions as the final actuation stage of the delivery chain. The perception subsystem determines when a target is valid, the navigation subsystem positions the robot appropriately, the feeding subsystem meters a single ball into place, and the launcher converts that prepared state into the final delivery action. This makes the launcher highly dependent on upstream subsystem performance, while also making it a key contributor to the final mission outcome.

Overall, the launcher subsystem emphasises compactness, response speed, and repeatability. Its design reflects the practical constraints of the TurtleBot3 platform while still providing sufficient performance for the mission's payload delivery task.

4.5 Electrical and Control Subsystem

The electrical subsystem focuses on controlling the on and off of the flywheel and the precise degree and timing of the servo motor while ensuring that the system is stable and has no damage to other subsystems. The flywheel should be off at all times and the servo motor should maintain at the starting position to prevent the ball dropping out. Once the "Perception Subsystem" detects the ArUco and identifies that the turtlebot is in position, the "done" signal will be sent such that the flywheel will start spinning. The servo motor will feed three balls according to the timing required (for station A). For the moving target (for station B), the servo motor will be controlled according to the ultrasonic sensor instead of timing. Once three balls are launched in each station, the turtlebot will go back to navigation mode and keep finding the next station.

Functions

The main functions of this subsystem are to:

- maintain the system electrical safety,
- receive serial communication from different subsystems,
- control the on and off of the flywheel precisely,
- control the angle of servo motor according to the timing requirement,
- use ultrasonic sensor to detect the moving object

Design Approach

To control the on and off of the flywheel, a N-type MOSFET is used. The gate of MOSFET is connected to the GPIO of Raspberry Pi. When the "Perception Subsystem" sends the "done" signal, the GPIO will be set to high, thus opening up the gate of MOSFET and the flywheel will start spinning. The PWM pin for the servo motor is connected to the GPIO of Raspberry Pi as well. Once the flywheel is at the designated speed, the PWM signal will be sent to the servo motor to turn 90 degrees to feed to balls. For the static target, the PWM signal is sent according to the required timing. For the moving target, the PWM signal is sent if the ultrasonic sensor detects the object between 17 and 23 centimeters.

After all three balls are launched, the "all launched" signal will be sent to turn off the gate of MOSFET by giving GPIO a low signal. The PWM signal will stop to make sure that the servo motor doesn't feed balls and prevent them from falling out. One important design concern is that when the gate is turned off, the back EMF from the flywheel might damage other components,

for example the power supply. To prevent that and maintain the electrical safety, a flyback wheel is used to prevent back EMF from damaging other components.

5. Interface Control Documents (ICD)

5.1 Hardware Interfaces

Table 1: Hardware Interface Specification

Interface	Source	Destination	Signal / Data Type	Purpose
LiDAR serial	2D LiDAR	Raspberry Pi	USB/ UART	SLAM and obstacle avoidance
CSI camera interface	Rpi camera	Raspberry Pi	MIPI CSI-2 video stream	ArUco detection
Main system serial	Raspberry Pi	OpenCR	USB/ UART	Odometry and IMU
GPIO 19 (PWM)	Raspberry Pi	Servo motor	PWM control signal	feed balls (control angles)
GPIO 23/24 (trigger/echo)	Raspberry Pi	Ultrasonic sensor	digital timing signal	detect the moving object
GPIO 18 (MOSFET gate control)	Raspberry Pi	Flywheel motor	digital high and low	control on and off of the flywheel
Motor control bus	OpenCR	TurtleBot motors	motion commands	drive and turning
Power lines	battery/ regulators	all components	DC power	component operation

The detailed connection is given in the Appendix B, which shows the specific USB, power and GPIO connection of the electrical components.

5.2 Software Interfaces

Table 2: Software Interfaces Specifications

Module	Input	Output	Description
SLAM node	LaserScan, odometry	Occupancy grid, robot pose	builds a 2D map, localises the robot, and the real time position
Exploration node	Occupancy grid, pose	next waypoint	analyze the map frontiers and generate the next waypoint to explore
Planner node	occupancy grid, target pose, waypoint	twist	computes global and local path, outputting the velocity commands for obstacles avoidance
Vision node	camera image	target detection result	identifies the ArUco and calculate the relative position
Launch controller	target state, distance data	servo and flywheel trigger	manages launch and feeding timing
Mission manager (FSM)	subsystem states	task selection/execution commands	coordinates overall mission, transection between “exploration” and “delivery”

5.3 Electrical Compatibility Notes

Each component must be connected according to its working voltage and control requirements. Voltage level compatibility is especially important for Raspberry Pi GPIO lines, servo motor supply, ultrasonic sensor operation, and motor driver interfaces. Improper grounding or mismatched logic levels may cause unstable operation and even break the Raspberry Pi due to the huge back current, so common grounding and protected interfacing should be maintained throughout the system. One key compatibility note is that there is a diode connecting on the sides of the flywheel to prevent the huge current flow back to Raspberry Pi when the flywheel stops.

6. Software / Firmware Development Documentation

6.1 Software Architecture

The software is divided into modular functional blocks:

- sensor acquisition,
- Slam mapping and localisation,
- Frontier exploration and coverage,
- ArUco target detection and tracking,
- Path planning and navigation,
- Mission state coordination
- Docking and alignment control
- Low-level motor actuation

This modular structure allows independent debugging and testing of perception, navigation, and actuation pipelines.

6.2 Main Control Logic

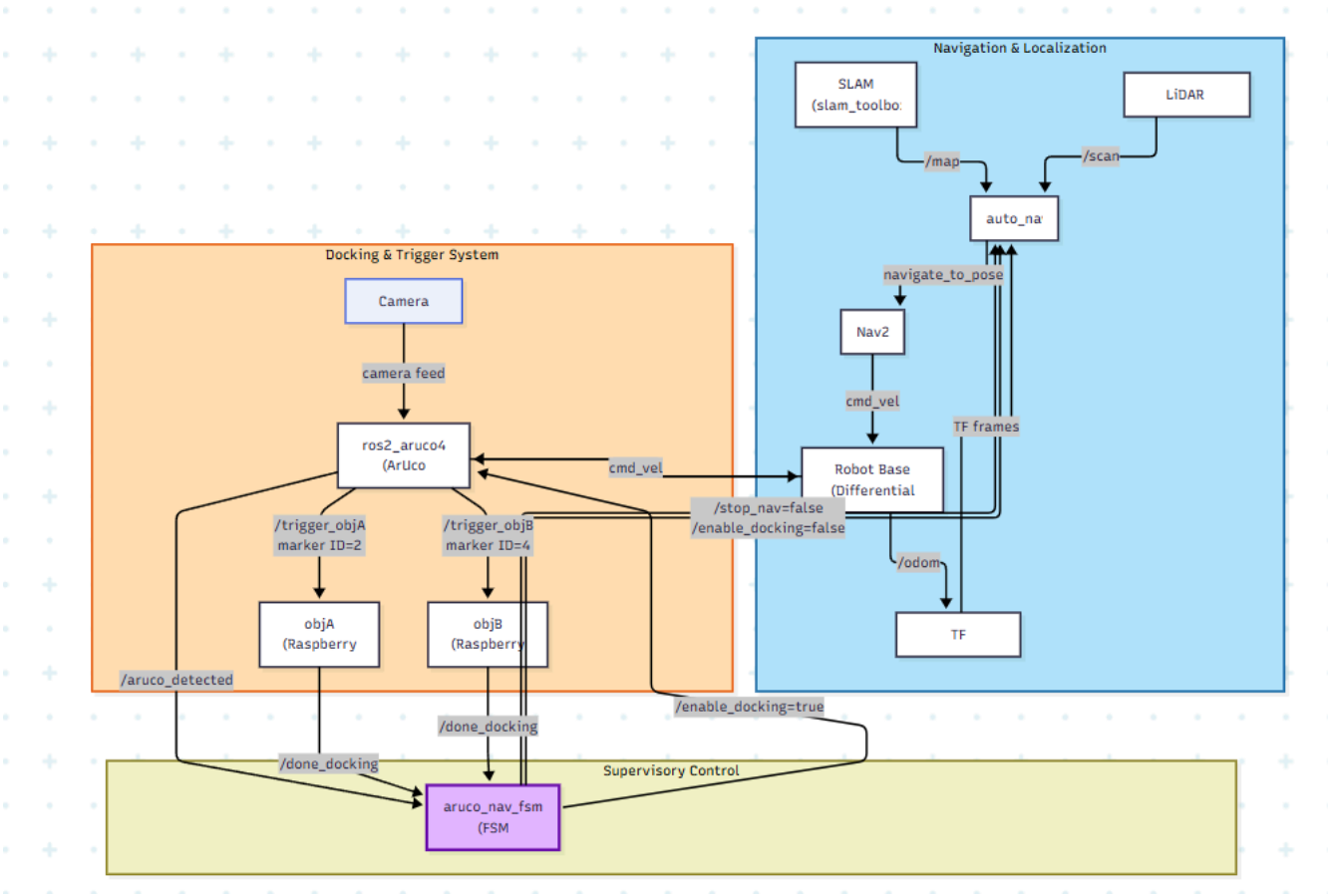


Figure 5: ROS Node Architecture

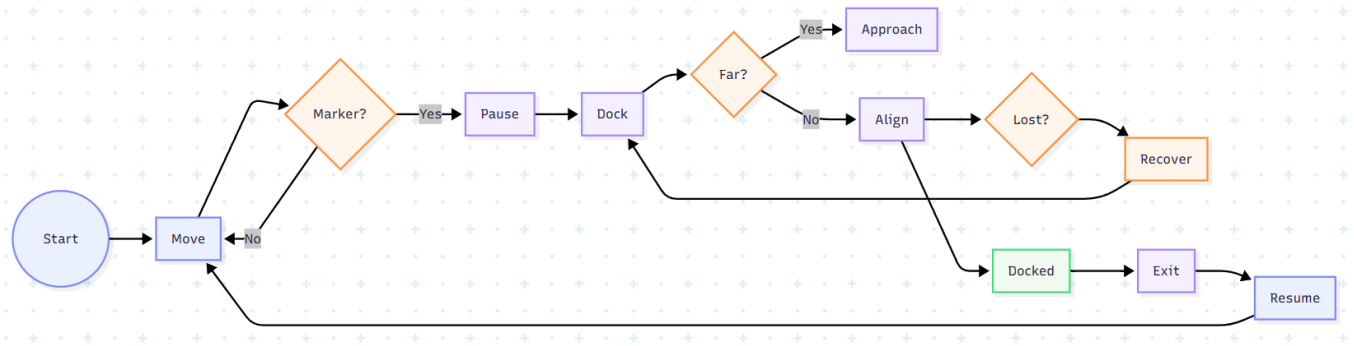


Figure 6: Core logic Diagram

The main control logic uses Nav2 as the primary navigation system for exploration and waypoint execution. The robot uses map data, TF-based localisation, and laser scan sensing to select exploration goals and send them to the Nav2 navigate_to_pose action server. When an ArUco marker is detected, a detection signal is published and monitored by the mission state machine. The FSM then switches the robot from navigation to docking by pausing Nav2 and enabling the docking controller.

Docking is carried out using a finite-state sequence. The robot locks onto the marker, estimates its relative pose from the camera image, aligns its orientation, corrects lateral offset, turns to face the marker, and then approaches until the required docking distance is reached. After successful docking, a trigger signal is published, and the FSM later returns control to Nav2 if exploration is still required.

6.3 Navigation Software

The navigation software uses SLAM-generated map data, TF-based localisation, frontier exploration, and the Nav2 stack for motion execution. Reachable exploration goals are selected and sent to Nav2, while local obstacle checks and recovery behaviour are applied using laser scan data. Once the map has been fully explored and no valid frontiers remain, the robot transitions from frontier exploration mode to coverage mode, where it systematically traverses the mapped environment to ensure complete area coverage. Nav2 continues to handle the main robot motion, while the FSM and docking controller take over only when Aruco marker is detected.

6.4 Payload Control Software

The launcher control logic can be separated into different parts depending on the type of the target: moving or non-moving. For the non-moving target, after the docking finishes, it will send a “done” signal. Once the “done” signal is received, the flywheel will start spinning for a few seconds until it reaches the targeted speed. Once it’s at the targeted speed, the servo will start feeding three balls according to the timing requirements. If all three balls are sent, the signal of “all launched” will be sent for turtlebot to complete the rest of the mission. For the moving target, the methodology is similar. Once the “done” signal is sent after finishing docking. The flywheel will start spinning as well, but the ball will only be fed by the servo motor if the ultrasonic sensor senses the target in the given distance. When all three balls are launched, the “all launched” signal is sent then the turtlebot will continue navigating.

6.5 Firmware / Hardware Integration

Firmware-level communication is required between the Raspberry Pi and actuators or motor controller interfaces. GPIO-based PWM is used for servo control. Serial or USB communication supports LiDAR and motor control pathways. Firmware must ensure timing consistency, safe startup sequencing, and proper command termination in case of failure.

6.6 Error Handling

The software should detect and handle:

- loss of localization,
- no valid path found,
- failed target detection,
- launch feed timeout,
- sensor communication failure,
- incomplete mission objective status.

Fallback behavior may include replanning, reattempting docking, re-scanning the environment, or returning to a known safe search pattern.

7. Real World Testing Documentation

7.1 Testing Objectives

Testing is carried out to ensure that each subsystem operates correctly in isolation and that the integrated system can reliably complete the mission. To minimize debugging time, each subsystem should be validated independently before system integration.

7.2 Subsystem Tests

Auto Mapping of Maze:

1. SLAM and Localisation
    a. Objective: Verify that the robot can generate a usable map in an unknown maze
    b. Method: Remotely control the robot and check if Cartographer map matches that of the physical layout
    c. Success Criteria: Map captures all obstacles, explored and unexplored areas, and accurate position of robot in the maze
2. Path Planning
    a. Objective: Verify that the robot can path to the nearest unexplored area
    b. Method: Monitor Rviz GUI as robot auto maps maze, check if end goal is unexplored, and if robot avoids collisions

c. Success Criteria: Maps to unexplored area without hitting any obstacles (bonus if able to prioritise biggest and nearest unexplored cluster)

ArUco Docking:

1. ArUco Recognition
    a. Objective: Robot is able to identify ArUco ID, position relative to the Rpi Camera
    b. Method: place ArUco and read logs of it's position relative to Rpi camera, as well as manually measuring the distance
    c. Success Criteria: Distance logged from Rpi Camera to ArUco is accurate
2. Docking
    a. Objective: Able to precisely and consistently dock in front of ArUco
    b. Method: Place Robot at different angles in front of ArUco, and monitor it's pathing, vision of ArUco and consistency of it's end point
    c. Success Criteria: Able to dock at the exact same spot from any angle

Payload Delivery:

1. Firing Mechanism
    a. Objective: Able to store up to 6 ping pong balls, and be fired into dropoff points
    b. Method: Place robot at ideal docking location and manually trigger firing of payload
    c. Success Criteria: No balls were stuck in flywheel, able to feed only 1 ball per fire, ball landed in dropoff point
2. Stationary Dropoff Point
    a. Objective: Dispense 3 ping pong balls into the dropoff point with precise timing intervals of 3 and 7 seconds
    b. Method: Place robot at ideal docking location and monitor timing intervals of ping pong balls with stopwatch
    c. Success Criteria: All 3 balls delivered with correct timing intervals
3. Moving Dropoff Point
    a. Objective: Dispense 3 ping pong balls into the moving dropoff point
    b. Method: Place robot at ideal docking location and monitor ultrasonic sensor output, as well as position of dropoff point when ping pong balls are fired
    c. Success Criteria: All ping pong balls delivered, regardless of speed of dropoff point and it's location when payload delivery begins

Integrated Mission Test:

1. Mapping and Docking
    a. Objective: Robot able to autonomously map out unknown maze, and upon finding ArUco path towards objective and dock precisely, while ignoring completed ArUcos
    b. Method: Place robot in unknown maze with ArUco’s scattered about
    c. Success Criteria: Able to map out the whole maze while prioritising docking of incomplete ArUcos. Docking itself is precise and consistent.

2. Docking and Firing
    a. Objective: Robot able to dock precisely and carry out the respective Objective Task
    b. Method: Place ArUco with it's respective dropoff point, place robot at any angle facing the ArUco
    c. Success Criteria: Able to dock consistently and precisely. Carry out respective objective with all balls landing in drop off point

7.3 Performance Metrics

Suggested metrics include:

- map completion time,
- localization drift,
- path efficiency,
- obstacle avoidance success rate,
- target detection accuracy,
- docking speed, and accuracy
- feed success rate,
- launch success rate,
- full mission completion rate.

7.4 Common Failure Modes to Monitor

Based on your earlier design reasoning, the main risks include:

- Missed out on unexplored areas
- Collision with obstacles during pathing
- Unable to detect ArUcos
- Unable to dock precisely
- Feeder jam
- Payload misses dropoff point
- Ultrasonic unable to detect moving dropoff point

7.5 Final Run Lessons Learnt

Mechanical redesign:

To address this, we decided to reduce the robot’s horizontal footprint mechanically. Over the course of about three hours, we restructured the robot using additional waffle plates and extended the structure vertically by two levels, with each extension using roughly half a plate width. This redesign allowed us to save significant horizontal space, and the final footprint became much closer to that of the original unmodified TurtleBot3. We did not make this design change earlier because we had initially believed that the maze clearance would be sufficient and that further tuning of Nav2 parameters would solve the issue. However, as time became limited, it became clear that sensor noise and real robot motion made that assumption unfeasible.

The main downside of this redesign was the lack of time for proper validation. After completing the restructuring, we had only about 30 minutes left to inspect the design and update the code parameters to account for the new dimensions, launcher position, launcher height, ultrasonic sensor position, camera position, as well as the overall stability and centre of gravity of the robot. Although most of these changes were accounted for correctly, some remaining mismatches caused issues during the actual run.

One such issue was the relative alignment between the ArUco marker reference and the launch hole. The offset between the camera and launcher should have been matched consistently to the offset between the target hole and the marker. Since this was not fully updated after the redesign, it led to one failed shooting attempt after we had already white-flagged. A second issue came from the reduction in support for the aluminium wire payload guide. In the original design, the wire structure used two support clamps and was intended to support seven ping pong balls. In the redesigned version, only one support clamp remained, and the structure was intended to carry six balls. During the actual run, the aluminium wire deformed slightly and opened up, releasing all of the payload at once. This was likely caused by a combination of motor vibration and the increased centre of gravity in the new structure. That failure then contributed to a reset in navigation during the first run.

If I had the opportunity to redesign the robot again, I would develop a more compact solution that fits fully within the waffle plate footprint between the OpenCR plate and the Raspberry Pi plate. I would design a more integrated printed mechanism within this space and increase the vertical spacing using stronger structural supports, such as printed hex supports or similar rigid members. Through this project, I also learned that the cost-to-value ratio of 3D printing for prototyping is actually very favourable. The cost of printing parts for a robot of this size is manageable, and the benefits in design freedom and iteration speed are significant. In hindsight, I would have preferred to print a more integrated mechanism with fewer, thicker, and more rigid combined parts. In the earlier prototypes, I intentionally split the system into many smaller parts to reduce printing volume because I overestimated the cost of printing. However, this resulted in a more fragmented and less robust structure. The rushed redesign also cost us some marks because the fasteners and structural stability were not as refined as they should have been.

Electrical System Improvement:

The original connection between Raspberry Pi and external electrical components were connected together via breadboard, especially the flywheel control circuit. However, we soon realize that connections on the breadboard are not stable due to two major reasons: the pin doesn’t fit the breadboard perfectly and the female connection on Raspberry Pi keeps coming out due to the vibration of Turtlebot when it’s moving. Sometimes the gate pin (GPIO 18) falls out such that the flywheel cannot start properly and only the servo motor is working.

To figure out the problem, we decided to solder the flywheel control circuit on the perfboard and use dupont housing to secure the connection on the Raspberry Pi. This improvement has brought multiple merits: the connections on the Raspberry Pi become more secure and it can prevent the short circuit between the neighboring pins, and the perfboard occupies less spaces compared to breadboard which makes the assembly easier.

Besides this issue, we also face the issue of lacking the 5V from Raspberry Pi. Raspberry Pi model 4 provides two 5V pins on the pinout, one is occupied by OpenCR, which provides the power supply to Raspberry Pi. Our original idea is to connect the power supply of all the external components onto 5V of Raspberry Pi. This will increase the number of loads quickly and increase the total current, which might potentially cause damage to Raspberry Pi. Therefore, we use 4 AA batteries as the

external power supply. In this way, the power supply can be more stable and the Raspberry will have less chance being damaged.

Software Drawbacks:

During the final run, the primary navigation failure occurred while the robot was traversing towards a valid unexplored frontier. This indicates that frontier selection was not the root cause of the issue, as the system was capable of identifying appropriate exploration targets. The failure instead occurred during path execution, when the robot became stuck at a ledge or constrained boundary region and was unable to recover. This is attributed to a combination of simplified planning assumptions, insufficient obstacle inflation, and limited recovery behaviour within the integrated system. Although the selected frontier was valid, the robot lacked the navigational robustness required to safely complete the path upon encountering a challenging region of the environment.

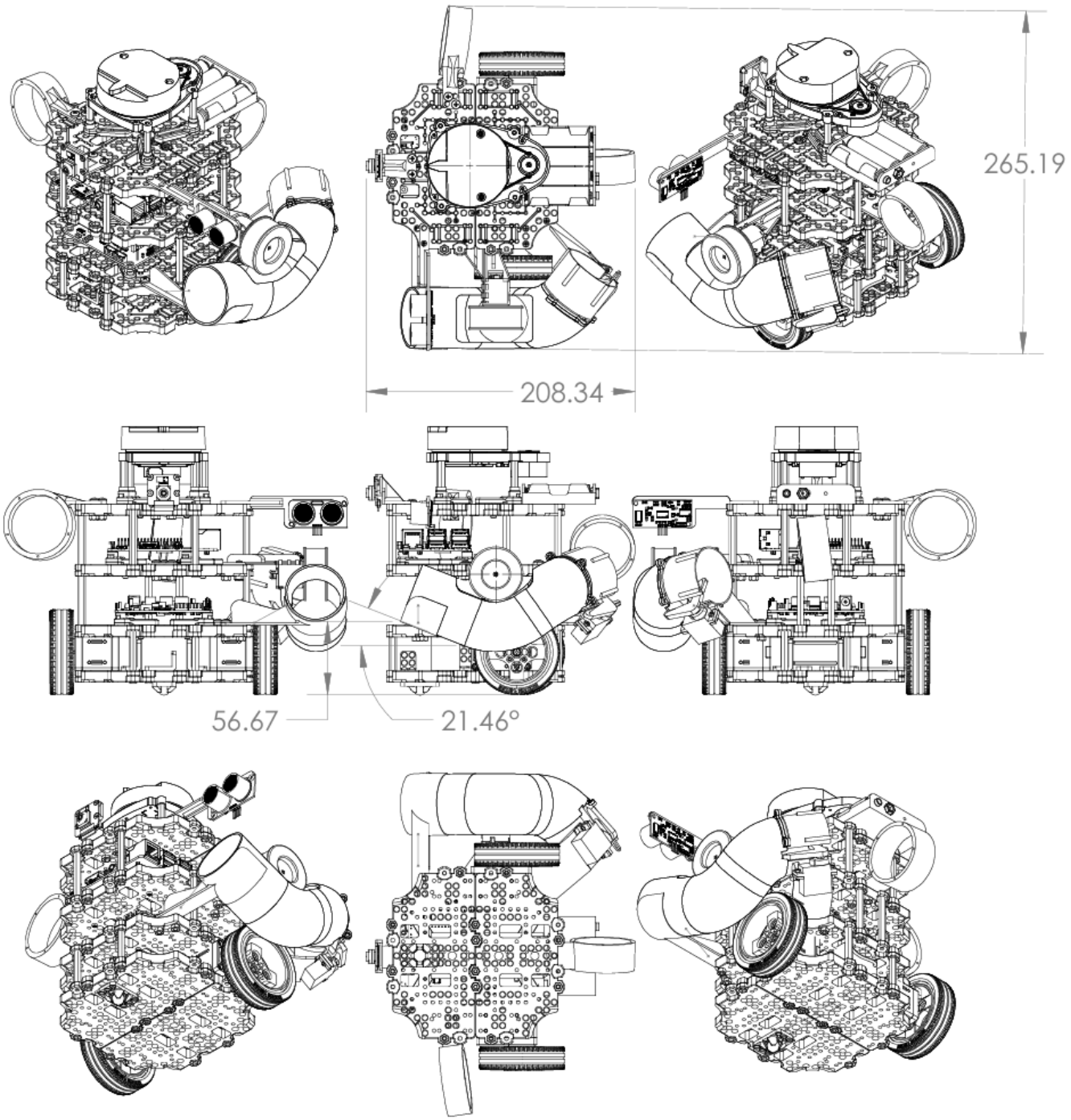
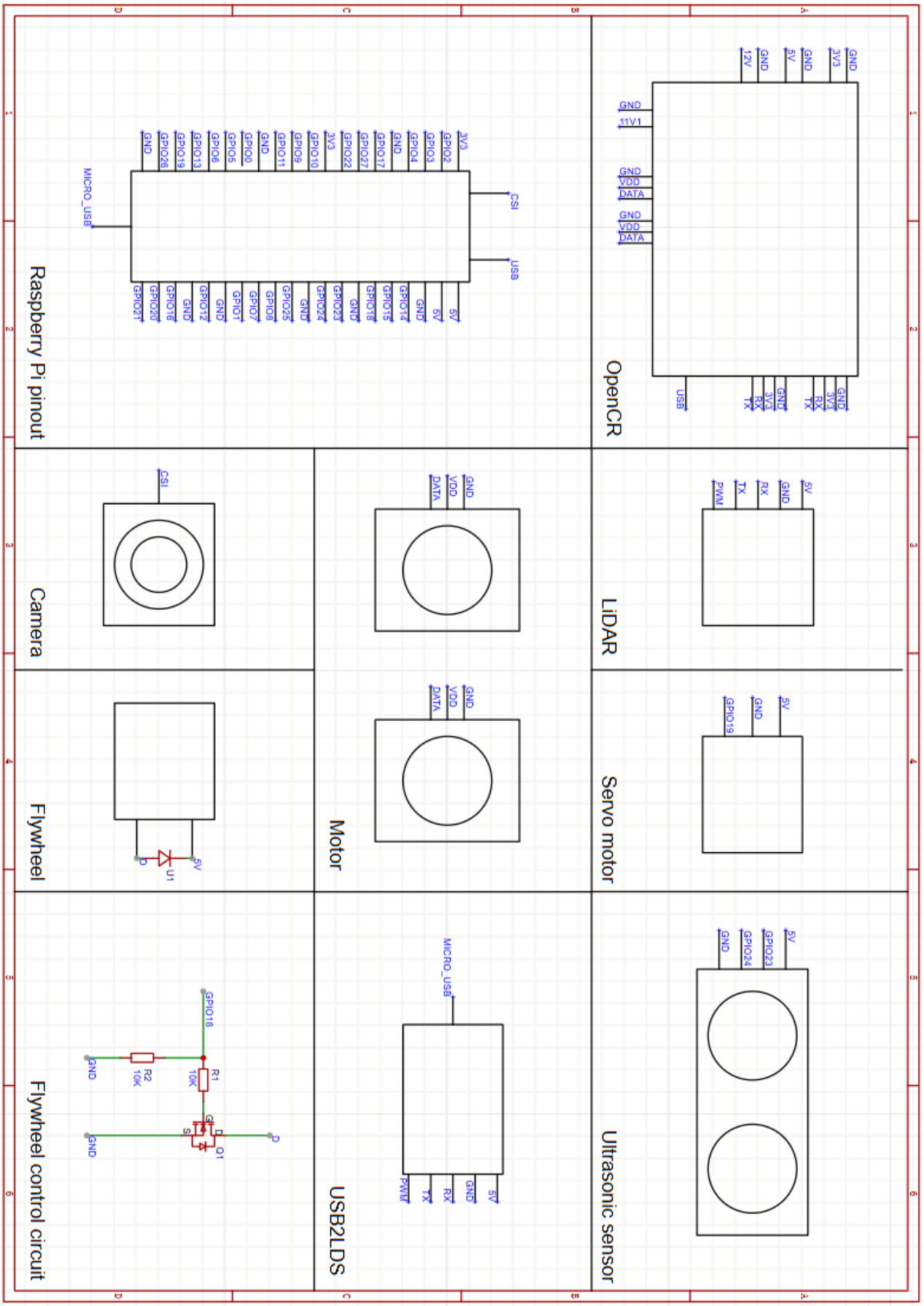
The docking failure presented a distinct set of issues. During isolated testing, the docking system performed correctly, demonstrating that the ArUco based localisation and approach logic were functional as a standalone module. The problem emerged only upon integration with the broader navigation stack. The most probable cause is that the ArUco marker pose used to initiate the docking sequence was obtained from a frame captured while the robot was still in motion. By the time the docking commands were executed, the robot had displaced from the position at which the pose was estimated, resulting in an approach based on an outdated relative transform. This caused the docking manoeuvre to be inaccurate despite the underlying logic being sound.

On reflection, the most significant limitation of this project was the insufficient time allocated to the integration between the autonomous navigation and docking subsystems. In hindsight, this integration represents the central objective of the module — developing individual components is of limited value if they cannot operate cohesively under a unified control architecture. By the time both subsystems were individually verified, insufficient time remained to properly address the handover logic, state synchronisation, and recovery mechanisms that robust integration requires. This is considered the most critical area for improvement in any future iteration of the system.

To address these shortcomings, several improvements are proposed for future work. With respect to navigation, obstacle inflation radii and safety margins should be carefully optimised, set conservatively enough to prevent the robot from navigating dangerously close to walls and ledges, yet not so aggressively that valid frontier regions become inaccessible or are prematurely closed off during exploration. Recovery behaviour should also be enhanced beyond simple reverse manoeuvres, incorporating strategies such as rotational escape, intermediate replanning from a safer pose, or broadening the obstacle detection zone to include lateral sectors. With respect to docking, the robot should be required to reach a complete stop and achieve a stable localisation estimate before the ArUco pose is computed.More fundamentally, the integration between navigation and docking must be treated as a primary design concern from the outset, with clearly defined state transition conditions, timing guarantees, and handover protocols to ensure that each subsystem operates on accurate and consistent state information.

8. Appendix

Appendix A: Mechanical Drawing of Design



Appendix B: Electrical Schematic Drawing of Design